Serial Python Minimalis:
Tutorial #05-Fungsi (Subprogram) dalam Python

Ahmad R. T. Nugraha

ver. 21 April 2026

Hingga saat ini, kode yang telah kita tulis belum banyak yang bisa digunakan ulang (*reusable*). Dalam pemrograman profesional, kita biasanya akan perlu mengelompokkan kode ke dalam blok-blok yang dapat diberi nama dan dieksekusi berulang kali (mungkin dengan parameter yang berbeda). Dengan kata lain, kita ingin menulis "subprogram" kita sendiri. Dalam bahasa Python, bentuk subprogram pada dasarnya adalah suatu "fungsi".

Penulisan fungsi tidak hanya berguna untuk meningkatkan portabilitas dan menghindari pengulangan baris kode, tetapi juga membantu memecah tugas-tugas kompleks menjadi tugas yang lebih sederhana. Dengan begitu, pada akhirnya kita dapat menghasilkan kode yang lebih mudah dipahami dan dikelola.

1 Pendefinisian Fungsi

Sebuah fungsi didefinisikan oleh ekspresi `def` yang memberinya nama dan menentukan "argumen" atau parameter apa saja yang diterimanya dalam tanda kurung, diikuti dengan titik dua (`:`). Badan fungsi diindentasi dengan 4 spasi. Saat dipanggil, segala pernyataan di dalam fungsi akan dieksekusi. Jika kata kunci `return` ditemui di titik mana pun, nilai yang ditentukan akan dikembalikan ke pemanggil (*caller*). Misalnya:

```python
def cube(x):
    x_cubed = x**3
    return x_cubed

cube(3)
```

Output:

```
27
```

```python
cube(4)
```

Output:

```
64
```

Fungsi di sini, `cube`, mengambil satu argumen, `x` (angka yang akan dipangkatkan tiga), dan mengembalikan satu nilai. Untuk mengembalikan dua nilai atau lebih dari sebuah fungsi, pisahkan dengan koma.

Program di bawah ini mendefinisikan sebuah fungsi untuk mengembalikan kedua akar dari persamaan kuadrat $ax^2 + bx + c$ (dengan asumsi persamaannya memiliki dua akar real). Parameter yang tepat yang diambil oleh fungsi ini adalah koefisien `a`, `b`, dan `c` yang mendefinisikan polinomial tersebut:

```
import math

def roots(a, b, c):
    d = b**2 - 4 * a * c
    r1 = (-b + math.sqrt(d)) / (2 * a)
    r2 = (-b - math.sqrt(d)) / (2 * a)
    return r1, r2

print(roots(1, 3, -10))
```

Output:

```
(2.0, -5.0)
```

2 Argumen *Keyword* dan *Default*

Sejauh ini, kita telah menggunakan argumen posisional dalam memanggil fungsi. Nilai yang akan ditetapkan ke setiap argumen fungsi ditentukan oleh urutan diberikannya nilai tersebut. Sebagai contoh, `roots(1, 3, -10)` menghasilkan pemanggilan fungsi `roots` dengan argumen `a=1`, `b=3`, dan `c=-10`. Kita juga dapat mengoper nilai ke sebuah fungsi dalam urutan sembarang menggunakan argumen *keyword*:

```
roots(b=3, c=-10, a=1)
```

Output:

```
(2.0, -5.0)
```

Argumen posisional dan *keyword* dapat digunakan dalam pemanggilan fungsi yang sama, tetapi argumen posisional harus didahulukan:

```
roots(1, c=-10, b=3)
```

Output:

```
(2.0, -5.0)
```

```
roots(1, b=3, -10)
```

Output:

```
  File "<ipython-input-10-b31133d01219>", line 1
    roots(1, b=3, -10)
                  ^
SyntaxError: positional argument follows keyword argument
```

Terkadang ada gunanya membuat argumen ke sebuah fungsi menjadi opsional. Dengan kata lain, kode fungsi akan menggunakan nilai bawaan (*default*) untuk argumen tersebut jika tidak ada nilai yang diberikan saat fungsi dipanggil. Hal ini dapat dilakukan di dalam definisi fungsi:

```
def Vm_ideal(T=298.15, p=101325):
    R = 8.31446 # konstanta gas, J.K-1.mol-1
    return R * T / p
```

Sekarang, fungsi `Vm_ideal` dapat dipanggil tanpa argumen sama sekali, dengan salah satu dari `T` atau `p`, atau dengan keduanya:

```
# Tanpa argumen: gunakan nilai default
Vm_ideal()
```

Output:

```
0.024465395993091537
```

```
# Gunakan T yang diberikan dan p default
Vm_ideal(273.15)
```

Output:

```
0.02241396248704663
```

```
# Gunakan T default dan p yang diberikan
Vm_ideal(p=1.e5)
```

Output:

```
0.02478956249
```

```
# Gunakan nilai yang diberikan untuk p dan T
Vm_ideal(600, 1.e3)
```

Output:

```
4.988676000000001
```

Kadang kita perlu juga mengoper isi dari sebuah `list` ke fungsi sebagai argumen-argumennya (daripada meneruskan `list` tersebut sebagai satu objek tunggal). "Sintaksis bintang" (*star syntax*) membongkar (*unpack*) suatu barisan dengan cara ini:

```
state = [298.15, 101325]
# Membongkar state menjadi argumen T, p
Vm_ideal(*state)
```

Output:

```
0.024465395993091537
```

3 Docstring

Docstring adalah literal string semacam komentar yang ditempatkan setelah *signature* (baris definisi) fungsi yang menjelaskan apa yang dilakukan fungsi tersebut, nilai apa yang dikembalikannya, dan argumen apa saja yang diterimanya. Jika string tersebut ditulis dengan tanda kutip tiga (`"""...."""` atau `'''...'''`), maka kita dapat memasukkan baris baru (*line breaks*).

```
def Vm_ideal(T=298.15, p=101325):
    """Return the molar volume of an ideal gas, in m^3.
    The temperature, T (in K) and pressure, p (in Pa) can be provided.
    """
    R = 8.31446 # gas constant, J.K-1.mol-1
    return R * T / p
```

Untuk memeriksa *docstring* sebuah fungsi dalam sesi Python interaktif (misalnya, IPython atau Jupyter notebook), ketik `help(func)` atau `func?`:

```
help(Vm_ideal)
```

Output:

```
Signature: Vm_ideal(T=298.15, p=101325)
Docstring:
Return the molar volume of an ideal gas, in m^3.
The temperature, T (in K) and pressure, p (in Pa) can be provided.
File:      <ipython-input...>
Type:      function
```

Teknik ini juga berlaku untuk fungsi bawaan dan fungsi yang diimpor:

```
help(math.asin)
```

Output:

```
Signature: math.asin(x, /)
Docstring:
Return the arc sine (measured in radians) of x.

The result is between -pi/2 and pi/2.
Type:      builtin_function_or_method
```

4 Cakupan (*Scope*)

Istilah cakupan (*scope*) mengacu pada cara nama variabel diselesaikan (*resolved*) dalam program Python, yaitu bagaimana interpreter Python mengetahui objek mana yang dirujuk oleh variabel tersebut. Pemahaman tentang cakupan sangat penting dalam pemrograman karena dimungkinkan untuk menggunakan nama variabel yang sama untuk merujuk pada objek yang berbeda di dalam dan di luar sebuah fungsi. Misalnya:

```
x, y = 1, 3

def add_y(z):
    x = y + z
    return x

add_y(x)
```

Output:

```
4
```

```
x
```

Output:

```
1
```

Minimalnya kita perlu memahami tiga jenis cakupan: (1) cakupan lokal (*local scope*), (2) cakupan global (*global scope*), dan (3) cakupan bawaan (*built-in scope*). Definisi `x, y = 1, 3` di atas membuat dua nama variabel dalam cakupan global, yakni sesuai namanya, variabel tersebut dapat diakses dari mana saja di dalam kode kita.

Definisi fungsi `add_y` mengambil satu argumen tunggal, yang diikat ke variabel baru bernama `z`. Badan fungsi memiliki akses ke objek ini serta objek `x` dan `y`. Namun, ketika penugasan `x = y + z` dibuat, bukan `x` global yang diperbarui, melainkan nama variabel baru yang juga disebut `x` yang telah dibuat dan bersifat lokal terhadap fungsi tersebut. Perubahan pada `x` di dalam fungsi ini tidak memengaruhi `x` global. Oleh karena itu, nilai yang dikembalikan dari fungsi adalah `4`, tetapi pemeriksaan nilai `x` setelah fungsi

mengembalikan nilainya akan memberikan nilai dari `x` global (yang tidak berubah) yaitu `1`.

Ada banyak hal yang terjadi di sini sehingga perlu dijelaskan sedikit lebih mendetail. Di dalam fungsi `add_y`, terdapat pernyataan penetapan atau penugasan, `x = y + z`. Penugasan dalam Python selalu dievaluasi dari kanan ke kiri (interpreter perlu mengetahui objek apa yang akan ditugaskan sebelum menyematkan nama variabel kepadanya). Oleh karena itu, ekspresi `y + z` harus dievaluasi, dan nilai `y` harus diperoleh (`z` dilewatkan sebagai argumen ke fungsi).

Aturan cakupan menentukan bagaimana nilai `y` diambil. Pertama, interpreter Python mencari di cakupan lokal fungsi, tetapi karena tidak ada variabel bernama `y` yang ada di cakupan ini, interpreter tidak menemukan apa pun. Selanjutnya, interpreter mencari di cakupan global kode (atau sesi Python interaktif) dan menemukan variabel bernama `y` yang tersemat pada bilangan bulat `3`. Jadi, sisi kanan dari pernyataan tersebut dievaluasi menjadi bilangan bulat `4`, dan Python diinstruksikan untuk menetapkan objek ini ke nama variabel `x`. Karena ini terjadi di dalam fungsi, nama variabel baru `x` dibuat dalam cakupan yang bersifat lokal terhadap fungsi tersebut. `x` inilah yang dirujuk dalam pernyataan `return`, dan karenanya objek `4` dikembalikan. Di luar fungsi, satu-satunya `x` adalah yang berada dalam cakupan global, dan itu tidak berubah.

Kita mungkin saja ingin benar-benar menaikkan (*increment*) bilangan bulat `x` dalam cakupan global, yang dengannya kita dapat menugaskan kembali objek yang dikembalikan oleh `add_y` ke `x`:

```
x = add_y(x)
x
```

Output:

```
4
```

Fungsi bukanlah satu-satunya tempat keberadaan cakupan lokal. Misalnya, *list comprehension* juga menggunakan cakupan lokal:

```
x = 1
[x**2 for x in range(6)]
```

Output:

```
[0, 1, 4, 9, 16, 25]
```

```
x
```

Output:

```
1
```

Cakupan bawaan (*built-in scope*) adalah tempat Python meletakkan kata kunci, fungsi bawaan, *exceptions*, dan atribut lain yang menyusun sintaksis dan fungsionalitas intinya.

Cakupan bawaaan adalah tempat terakhir yang dicari oleh *interpreter* saat mencoba menyelesaikan (*resolve*) sebuah nama variabel. Misalnya, jika kita (secara tidak bijak) mengikat nama `len` ke sebuah objek, nama ini tidak akan lagi merujuk pada fungsi bawaan `len()` (untuk mencari panjang suatu urutan):

```
len('abc')
```

Output:

```
3
```

```
len = 100 # diperbolehkan; tapi bukan ide yang bagus.
len('abcdef')
```

Output:

```
TypeError                                 Traceback (most recent call last)
<ipython-input-3-4f6544af4f86> in <module>
----> 1 len('abcdef')

TypeError: 'int' object is not callable
```

Pada penggunaan pertamanya, nama `len` hanya ditemukan di cakupan bawaan sebagai fungsi panjang (*length function*), dan begitulah cara penggunaannya. Setelah `len` ditetapkan ke bilangan bulat `100`, pendefinisian ini eksis dalam cakupan global dan ditemukan di sana sebelum dicari di cakupan bawaan, sehingga penggunaannya sebagai sebuah fungsi memunculkan *exception*, yakni `len('abcdef')` mencoba memanggil objek bilangan bulat `100` dengan sebuah argumen, `'abcdef'`, yang merupakan suatu kesalahan (*error*).

5 Fungsi `lambda`

Ada kalanya berguna juga bagi kita untuk mendefinisikan dan menggunakan fungsi sederhana, tetapi tanpa membungkusnya dalam blok `def` atau memberinya nama. Pada kasus ini, fungsi `lambda` dapat memainkan peranan penting. Sintaksis untuk mendefinisikan fungsi `lambda` adalah:

```
lambda arguments: expression
```

dengan `arguments` adalah urutan argumen yang dipisahkan koma, seperti fungsi biasa, dan `expression` adalah ekspresi Python sederhana (tidak diperbolehkan adanya blok perulangan atau pengondisian). Misalnya, dua hal berikut ini ekuivalen:

```
def adder(a, b):
    return a + b
```

```
adder(5, 6)
```

Output:

```
11
```

```
adder = lambda a, b: a + b
adder(5, 6)
```

Output:

```
11
```

Pada praktiknya, fungsi `lambda` bisa dibilang agak terlalu sering disalahgunakan, dan biasanya jika fungsi `lambda` diikat ke sebuah nama variabel seperti ini (yang tidak lagi menjadi anonim), fungsi `def` reguler lebih disarankan. Namun, fungsi `lambda` sangat berguna dalam kasus-kasus sekali pakai (*one-off cases*) ketika objek fungsi itu sendiri diteruskan ke fungsi lain. Contoh yang umum adalah dalam mengurutkan sebuah `list` dari `tuple`. Tinjau `list` berikut yang berisi tiga `tuple`, masing-masing menyimpan simbol unsur dan nomor massanya:

```
masses = [('H', 1), ('Ar', 40), ('C', 12)]
```

Pengurutan `list` ini dengan metode `sort` akan menghasilkan `list tuple` yang terurut berdasarkan urutan abjad dari item pertama masing-masing:

```
masses.sort()
masses
```

Output:

```
[('Ar', 40), ('C', 12), ('H', 1)]
```

Misalkan kita ingin mengurutkan `list` tersebut berdasarkan item kedua dari setiap `tuple`, yaitu nomor massanya. Metode `sort` menerima argumen opsional, `key`, yang mengharapkan untuk menerima sebuah fungsi. Fungsi ini diteruskan ke setiap item di dalam `list`, dan `list` tersebut diurutkan berdasarkan apa pun yang dikembalikan oleh fungsi itu untuk setiap item. Kita tinjau contoh berikut ini.

```
def get_mass(e):
    return e[1]

masses.sort(key=get_mass)
masses
```

Output:

```
[('H', 1), ('C', 12), ('Ar', 40)]
```

Perhatikan sintaksisnya: `key` harus berupa sebuah fungsi, argumen tunggalnya adalah sebuah item dari `list` (di sini, dipanggil `e`, tetapi nama apa pun boleh). Fungsi bernama `get_mass` ini memiliki tugas sederhana untuk mengembalikan item kedua dari `tuple` yang diteruskan kepadanya. Namun, karena ini mungkin satu-satunya tempat fungsi semacam itu digunakan, lebih mudah untuk menggunakan fungsi `lambda` secara sejajar (*in-line*):

```
masses = [('H', 1), ('Ar', 40), ('C', 12)]
masses.sort(key=lambda e: e[1])
masses
```

Output:

```
[('H', 1), ('C', 12), ('Ar', 40)]
```

Demikian pula, pengurutan string dengan mengabaikan spasi di awal (*leading whitespace*) dapat dicapai dengan menggunakan `strip()` di dalam fungsi `lambda`:

```
sorted([' alpha', 'beta', ' gamma'])
```

Output:

```
[' gamma', ' alpha', 'beta']
# spasi diikutkan dalam pengurutan
```

```
# Mengurutkan berdasarkan string tanpa spasi di awal.
sorted([' alpha', 'beta', ' gamma'], key=lambda s: s.strip())
```

Output:

```
['alpha', 'beta', ' gamma']
```

Sebenarnya, dalam kasus ini, kita tidak memerlukan fungsi `lambda` karena metode string `strip` tersedia langsung dari objek tipe `str`:

```
sorted([' alpha', 'beta', ' gamma'], key=str.strip)
```

Output:

```
['alpha', 'beta', ' gamma']
```

Studi Kasus

Kasus 1 Kapasitas Panas Einstein

Model Einstein untuk kapasitas panas dari sebuah kristal adalah

$$C_{V,m} = 3R\left(\frac{\Theta_E}{T}\right)^2 \frac{\exp\left(\frac{\Theta_E}{T}\right)}{\left[\exp\left(\frac{\Theta_E}{T}\right) - 1\right]^2}$$

dengan suhu Einstein, Θ_E, adalah konstanta untuk material tertentu.
Tuliskan sebuah fungsi Python untuk menghitung $C_{V,m}$ pada 300 K untuk (a) natrium ($\Theta_E = 192$ K) dan (b) intan ($\Theta_E = 1450$ K).

Solusi

Fungsi `CV_Einstein` mengambil dua argumen: `T`, suhu tempat menghitung kapasitas panas; dan `ThetaE`, suhu karakteristik Einstein untuk material tersebut.

```python
import numpy as np

def CV_Einstein(T, ThetaE):
    """Return the Einstein molar heat capacity of a material, C_Vm.
    T is the temperature (K) and ThetaE the Einstein temperature for
    ↪ the
    material being considered. C_Vm is returned in J.K-1.mol-1.
    """
    # Konstanta gas, J.K-1.mol-1
    R = 8.314462618
    x = ThetaE / T
    f = np.exp(x)
    return 3 * R * f * (x / (f - 1))**2

T = 300
ThetaE_Na, ThetaE_dia = 192, 1450

CV_Na = CV_Einstein(T, ThetaE_Na)
CV_dia = CV_Einstein(T, ThetaE_dia)

print(f'Material CVm({T} K)')
print('{:7s} {:8.2f} J.K-1.mol-1'.format('Na', CV_Na))
print('{:7s} {:8.2f} J.K-1.mol-1'.format('diamond', CV_dia))
```

Output:

```
Material CVm(300 K)
Na          24.11 J.K-1.mol-1
diamond      4.71 J.K-1.mol-1
```

Kasus 2 Persamaan Keadaan Gas

Untuk persamaan-persamaan keadaan berikut, definisikan fungsi Python yang memberikan tekanan gas, p, jika diberikan volume V, jumlah mol n, dan suhu T.
Persamaan keadaan gas ideal:

$$p = \frac{nRT}{V}$$

Persamaan keadaan van der Waals:

$$p = \frac{nRT}{V - nb} - \frac{n^2 a}{V^2}$$

Persamaan keadaan Dieterici:

$$p = \frac{nRT \exp\left(-\frac{na}{RTV}\right)}{V - nb}$$

Bandingkan tekanan yang diprediksi untuk 1 mol CO_2 pada $T = 273.15$ K yang dibatasi dalam volume 20 L. Ambil parameter van der Waals $a = 3.640\ \mathrm{L^2\ bar\ mol^{-2}}$ dan $b = 0.04267\ \mathrm{L\ mol^{-1}}$; dan parameter Dieterici $a = 4.692\ \mathrm{L^2\ bar\ mol^{-2}}$ dan $b = 0.04639\ \mathrm{L\ mol^{-1}}$.
Ulangi perhitungan tersebut untuk jumlah 0.01 mol.

Solusi

Pertama, definisikan fungsi Python untuk masing-masing persamaan keadaan.

```python
import numpy as np

# Konstanta gas, J.K-1.mol-1
R = 8.314462618

def p_ideal(n, T, V):
    """Ideal gas equation of state, SI units."""
    return n * R * T / V

def p_vdw(n, T, V, a, b):
    """Van der Waals equation of state, SI units."""
    return n * R * T / (V - n * b) - n**2 * a / V**2

def p_dieterici(n, T, V, a, b):
    """Dieterici equation of state, SI units."""
    return n * R * T * np.exp(-n * a / (R * T * V)) / (V - n * b)
```

Sekarang panggil fungsi-fungsi tersebut dengan variabel keadaan yang sesuai, `n`, `T`, dan `V` serta parameter `a` dan `b`. Berhati-hatilah untuk mengonversi semuanya ke dalam satuan SI.

```python
# Jumlah gas (mol), suhu (K), dan volume (m3).
n, T, V = 1, 273.15, 20 * 1.e-3

print(f'n = {n} mol, T = {T} K, V = {V} m3')

# Persamaan keadaan gas ideal.
print(f'Ideal:         p = {p_ideal(n, T, V):.3f} Pa')

# Persamaan keadaan Van der Waals (a dikonversi ke m6.Pa.mol-2, b ke
↪  m3.mol-1).
a_vdw, b_vdw = 3.640 / 10, 0.04267 / 1.e3
print(f'Van der Waals: p = {p_vdw(n, T, V, a_vdw, b_vdw):.3f} Pa')

# Persamaan keadaan Dieterici (a dikonversi ke m6.Pa.mol-2, b ke
↪  m3.mol-1).
a_diet, b_diet = 4.692 / 10, 0.04639 / 1.e3
print(f'Dieterici:     p = {p_dieterici(n, T, V, a_diet, b_diet):.3f}
↪  Pa')
```

Output:

```
n = 1 mol, T = 273.15 K, V = 0.02 m3
Ideal:         p = 113554.773 Pa
Van der Waals: p = 112887.560 Pa
Dieterici:     p = 112649.100 Pa
```

Sekarang ulangi perhitungan tersebut untuk 0.01 mol:

```
n = 0.01
print(f'n = {n} mol, T = {T} K, V = {V} m3')
print(f'Ideal:         p = {p_ideal(n, T, V):.3f} Pa')
print(f'Van der Waals: p = {p_vdw(n, T, V, a_vdw, b_vdw):.3f} Pa')
print(f'Dieterici:     p = {p_dieterici(n, T, V, a_diet, b_diet):.3f}
↪  Pa')
```

Output:

```
n = 0.01 mol, T = 273.15 K, V = 0.02 m3
Ideal:         p = 1135.548 Pa
Van der Waals: p = 1135.481 Pa
Dieterici:     p = 1135.457 Pa
```

Pada tekanan yang lebih rendah (dicapai dengan membatasi jumlah gas yang lebih sedikit pada volume dan suhu yang sama dengan sebelumnya), ketiga persamaan keadaan tersebut memberikan nilai-nilai yang hampir sama.